

Advanced: Arrays and ...

Try  [HackMD \(https://hackmd.io?utm_source=view-page&utm_medium=logo-nav\)](https://hackmd.io?utm_source=view-page&utm_medium=logo-nav).

Arrays 1: One Dimensional

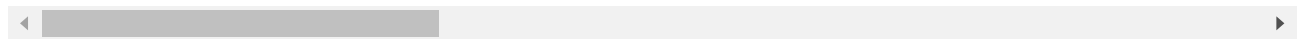
Problem 1 - Find Maximum Subarray Sum

Problem Statement

Given an integer array A, find the maximum subarray sum out of all the subarrays.

How other than this, can Kadane's be asked ?

Imagine you're a financial analyst tracking the daily stock prices of a company. In this scenario, you want to determine the duration (number of consecutive days



Examples

Example 1:

For the given array A with length N,

Index	0	1	2	3	4	5	6
Array	-2	3	4	-1	5	-10	7

Output:

Max Sum: 11

Subarray: 3 4 -1 5

Optimized Solution - Kadanes Algorithm

Solution:

Say we maintain two variables, max_sum and cur_sum. max_sum keeps track of maximum sum found till now and cur_sum keeps track of the current sum.

We will consider elements one by one. If there's a positive element, we can just add it to our currsum, but a negative element is considered only if the overall sum is positive because in the future if positives come, they may further increase this positivity(sum).

If after adding a negative element, the curr_sum becomes < 0 , then we shall not carry it forward since it will further decrease any number that'll be added to it, curr_sum will be set to 0 in this case.

Example -

$A[] = \{-2, 3, 4, -1, 5, -10, 7\}$

Answer array: 3, 4, -1, 5

Explanation:

$$3 + 4 = 7$$

$$7 + (-1) = 6 \text{ (still positive)}$$

$$6 + 5 = 11 \text{ (higher than 7)}$$

Dry Run

$\begin{matrix} 0 & 1 & 2 & 3 & 4 & 5 & 6 & 7 & 8 \\ \{ -20, & 10, & -20, & -12, & 6, & 5, & -3, & 8, & -2 \} \end{matrix}$

i	A[i]	currSum	maxSum	
0	-20	-20	-20	reset the currSum = 0 and do not take it forward since adding a negative will make it more negative and adding a positive will reduce positivity of that element.
1	10	10	10	
2	-20	$10 + (-20) = -10$	10	reset currSum = 0
3	-12	-12	10	reset currSum = 0
4	6	6	10	
5	5	$6 + 5$	11	
6	-3	$11 - 3 = 8$	11	Keep currSum as 8 only since if we find a positive, it can increase the sum
7	8	$8 + 8 = 16$	16	
8	-2	$16 - 2 = 14$	16	Keep currSum as 14 only since if we find a positive, it can increase the sum

Final maxSum = 16

Pseudocode

```
int maximumSubarraySum(int[] arr, int n) {  
    int maxSum = Integer.MIN_VALUE, currSum = 0;  
  
    for (int i = 0; i <= n - 1; i++) {  
        currSum += arr[i];  
  
        if (currSum > maxSum) {  
            maxSum = currSum;  
        }  
  
        if (currSum < 0) {  
            currSum = 0;  
        }  
    }  
  
    return maxSum;  
}
```

Time and Space Complexity

TC - $O(n)$

SC - $O(1)$

Problem 2 - Perform multiple Queries from index i to j

Given an integer array A such that all the elements in the array are 0. Return the final array after performing multiple queries

Query: (i, j, x) : Add x to all the elements from index i to j

Given that $i \leq j$

Example:

Let's take an example, say we have the zero-filled array of size 7 and the queries are given as

q1 = (1, 3, 2)

q2 = (2, 5, 3)

q3 = (5, 6, -1)

Index	0	1	2	3	4	5	6
Arr[7]	0	0	0	0	0	0	0
V1		2	2	2			
V2			3	3	3	3	
V3						-1	-1
Ans	0	2	5	5	3	2	-1

Optimized Solution

Idea

- We can add the value at the starting index and subtract the same value just after the ending index which will help us to only carry the effect of **+val** till a specific index.
- Following the index(k) where we have done **-val**, the effect will neutralise.

Pseudocode:

```
zeroQ(int N, int start[ ], int end[ ], int val[ ]){
    long arr[N] = 0;
    for(int i = 0; i < Q; i++){

        //ith query information: start[i], end[i], val[i]
        int s = start[i], e = end[i], v = val[i];

        arr[s] = arr[s] + v;

        if(e < n - 1){
            arr[e + 1] = arr[e + 1] - v;
        }
    }

    //Apply cumm sum a psum[] on arr
    for (i = 1; i < N; i++){
        arr[i] += arr[i - 1];
    }

    return arr;
}
```

Time and Space complexity for optimised solution

TC: $O(Q + N)$

SC: $O(1)$

Problem 3 - Rain Water Trapping

Given N buildings with height of each building, find the rain water trapped between the buildings.

Example Explanation

Example:

`arr[] = {2, 1, 3, 2, 1, 2, 4, 3, 2, 1, 3, 1}`

We now need to find the rainwater trapped between the buildings

Ans: 8

Idea:

If we get units of water stored over every building, then we can get the overall water by summing individual answers.

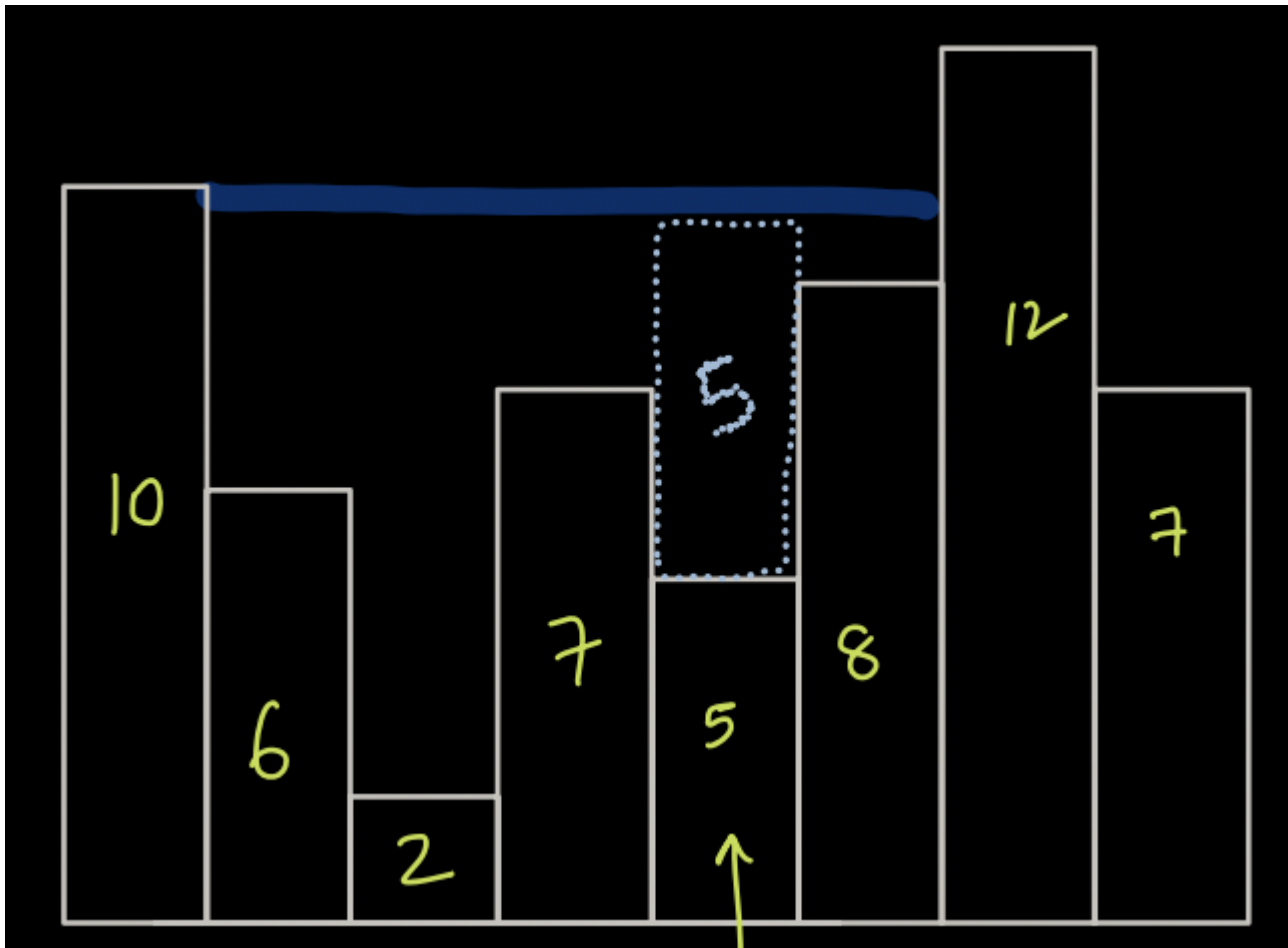
Water over every building depends on the height of the minimum of the largest buildings on either sides.

Example:

Water stored over building 5 depends on minimum of the largest building on either sides.

i.e, $\min(10, 12) = 10$

Water stored over 5 is $10 - 5 = 5$ units.



Optimised Approach

We can store the max on right & left using carry forward approach.

- We can take 2 arrays, lmax[] & rmax[].
- Below is the calculation for finding max on left & right using carry forward approach.
- This way, we don't have to find max for every element, as it has been pre-calculated.

	4	2	5	7	4	2	3	6	8	2	3
lmax[]:	0	4	4	5	7	7	7	7	7	8	8
rmax[]:	8	8	8	8	8	8	8	8	3	3	0

Pseudocode

```
ans = 0;

int lmax[N] = {0};
for(int i = 1; i < N; i++) {
    lmax[i] = max(lmax[i - 1], A[i - 1]);
}

int rmax[N] = {0};
for(int i = N - 2; i >= 0; i--) {
    rmax[i] = rmax[i + 1], A[i + 1];
}

for(int i = 1; i < N - 1; i++) {
    water = min(lmax[i], rmax[i]) - A[i];

    if(water > 0) {
        ans += water;
    }
}
```

Time & Space

Time - $O(N)$ {Since we have precalculated lmax & rmax}

Space - $O(N)$

Advanced DSA : Arrays 2: Two Dimensional

Problem 1 - Search in rowwise and colwise sorted matrix

Given a row wise and column wise sorted matrix, find out whether element **k** is present or not.

Example

Observe that rows and columns are both sorted.

A =

	-5	-2	1	13
	-4	0	3	14
	-3	2	6	18

Test Case 1

13 => Present (true)

Test Case 2

15 => Not Present (false)

Idea

- We shall exploit the property of the matrix being sorted.
- Start with the cell from where we can decide the next step.

Example:

-5	-2	1	13
-4	0	3	14
-3	2	6	18

Search for: 0

Say we stand at **top right cell i.e, 13**.

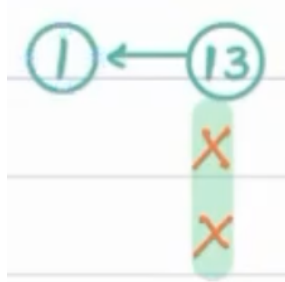
Now, **13 > 0**, should we go left or down ? Can we decide ?

Yes, if we move down the elements are > 13, but we are looking for an element < 13, so we should move left.

It means, all elements below 13, can be neglected.

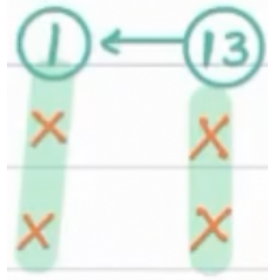


Move Left

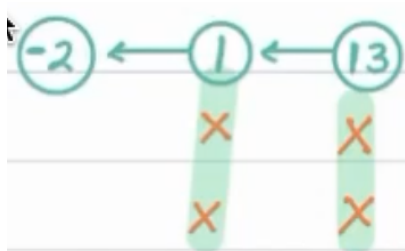


Now, where shall we move ?

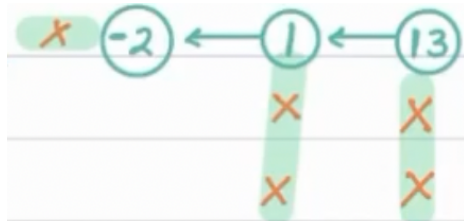
Since, $1 > 0$, again all elements below 1 are greater than 1, hence can be neglected.



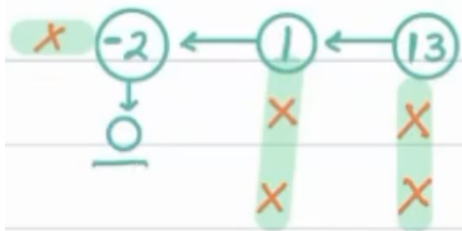
Move Left



Now, $-2 < 0$, all elements on left of -2 are lesser than -2, hence can be neglected.



Move Down



Approach

- We can start at top right cell.
- If $A[i][j] < K$, move down, else move left.
- Repeat until the element is found, or the search space is exhausted.

NOTE: We could have also started at bottom left cell.

Pseudocode

```

1  int i = 0, j = M - 1
2  while(i < N && j >= 0){
3      if(arr[i][j] == K){
4          return true;
5      } else if(arr[i][j] < K){
6          i++; //move down; next row
7      } else{
8          j--; //move left; previous column
9      }
10 }
11 return false;

```

Time & Space Complexity

TC: $O(M+N)$ since at every step, we are either discarding a row or a column. Since total rows+columns are $N+M$, hence iterations will be $N+M$.

SC: $O(1)$

Problem 2 - Print Boundary Elements

Given an matrix of $N \times N$ i.e. $Mat[N][N]$, print boundary elements in clockwise direction.

Example:

$N = 5$

1	2	3	4	5
6	7	8	9	10
11	12	13	14	15
16	17	18	19	20
21	22	23	24	25

Output: [1, 2, 3, 4, 5, 10, 15, 20, 25, 24, 23, 22, 21, 16, 11, 6]

Approach

- Print N-1 elements of first row from left to right
- Print N-1 elements of last column from top to bottom
- Print N-1 elements of last row from right to left
- Print N-1 elements of first column from bottom to top

Pseudocode

```
function printBoundaryElements(Mat[][],N) {
    i=0 j=0

    for(idx = 1 ; idx < N ; idx++){
        print(Mat[i][j] + ",")
        j++
    }

    for(idx = 1 ; idx < N ; idx++){
        print(Mat[i][j] + ",")
        i++
    }

    for(idx = 1 ; idx < N ; idx++){
        print(Mat[i][j] + ",")
        j--
    }

    for(idx = 1 ; idx < N ; idx++){
        print(Mat[i][j] + ",")
        i--
    }
}
```

Problem 3 - Spiral Matrix

Given an matrix of $N \times N$ i.e. $\text{Mat}[N][N]$. Print elements in spiral order in clockwise direction.

Example

$N = 5$

	0	1	2	3	4	5
0	1	2	3	4	5	6
1	7	8	9	10	11	12
2	13	14	15	16	17	18
3	19	20	21	22	23	24
4	25	26	27	28	29	30
5	31	32	33	34	35	36

Here is the depiction to understand the problem better:

	0	1	2	3	4	5
0	1	2	3	4	5	6
1	7	8	9	10	11	12
2	13	14	15	16	17	18
3	19	20	21	22	23	24
4	25	26	27	28	29	30
5	31	32	33	34	35	36

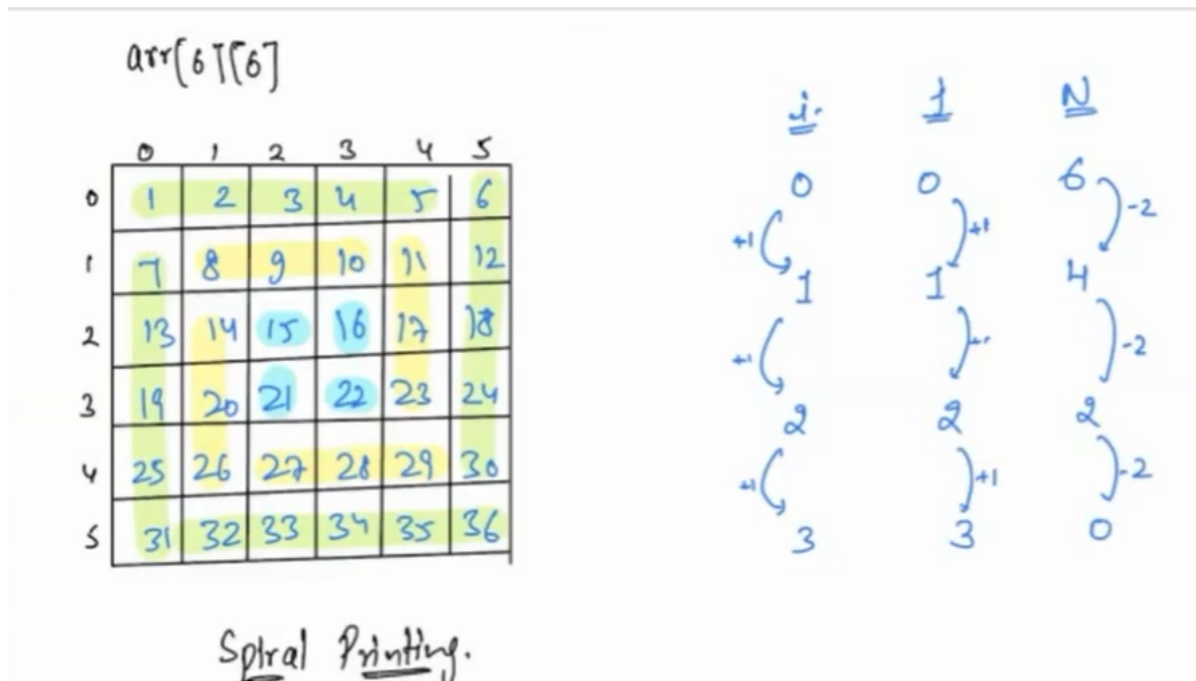
Solution = [1,2,3,4,5,6,12,18,24,30,36,35,34,33,32,31,25,19,13,7,8,9,10,11,17,23]

The red arrow represents direction of traversal(clockwise) and fashion in which elements are traversed.

Approach

- We can break the problem into several boundary printing problem discussed above

- So first print boundary of matrix of $N \times N$
- Then we print boundary of next submatrix with top left element being (1,1) and Bottom right element being (N-2,N-2).
- After every boundary, to print the next boundary, N will be reduced by 2 and i & j will be incremented by 1.
- We do this till matrix of size least size is reached.



Boundaries of submatrices are highlighted in different color.

Edge Case

Will the above code work if matrix size is 1 ?

No, since the loops run $N-1$ times, therefore we have to handle it separately.

Problem 4 - Sum of all Submatrices Sum

Given a matrix of N rows and M columns determine the sum of all the possible submatrices.

Example:

	0	1	2
0	4	9	6
1	5	-1	2

All Possible sub-matrices are -

$$\begin{array}{llll}
 \begin{bmatrix} 4 \end{bmatrix} \rightarrow 4 & \begin{bmatrix} 4 & 9 \end{bmatrix} \rightarrow 13 & \begin{bmatrix} 4 \\ 5 \end{bmatrix} \rightarrow 9 & \begin{bmatrix} 4 & 9 \\ 5 & -1 \end{bmatrix} \rightarrow 17 \\
 \begin{bmatrix} 9 \end{bmatrix} \rightarrow 9 & \begin{bmatrix} 9 & 6 \end{bmatrix} \rightarrow 17 & \begin{bmatrix} 9 \\ -1 \end{bmatrix} \rightarrow 8 & \begin{bmatrix} 9 & 6 \\ -1 & 2 \end{bmatrix} \rightarrow 16 \\
 \begin{bmatrix} 6 \end{bmatrix} \rightarrow 6 & \begin{bmatrix} 5 & -1 \end{bmatrix} \rightarrow 4 & & \\
 \begin{bmatrix} 5 \end{bmatrix} \rightarrow 5 & \begin{bmatrix} -1 & 2 \end{bmatrix} \rightarrow 1 & \begin{bmatrix} 6 \\ 2 \end{bmatrix} \rightarrow 8 & \\
 \begin{bmatrix} -1 \end{bmatrix} \rightarrow -1 & \begin{bmatrix} 4 & 9 & 6 \end{bmatrix} \rightarrow 19 & & \begin{bmatrix} 4 & 9 & 6 \\ 5 & -1 & 2 \end{bmatrix} \rightarrow 25 \\
 \begin{bmatrix} 2 \end{bmatrix} \rightarrow 2 & \begin{bmatrix} 5 & -1 & 2 \end{bmatrix} \rightarrow 6 & &
 \end{array}$$

Total Sum = 166

Approach

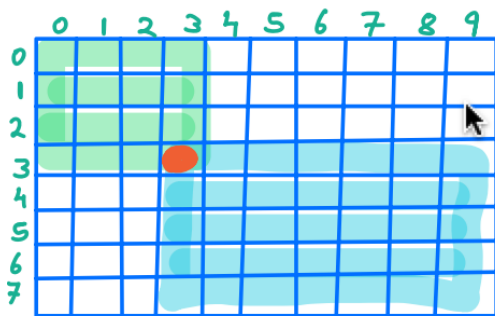
This question can be solved using **contribution technique**.

We will find that in how many submatrices a particular element is part of, then we just have to add up the individual results.

In what all submatrices, a particular element is part of ?

Let's look at the red cell in below figure.

If we combine all the top left cells (marked with green color) with all the bottom right cells (marked with blue color), then in all those submatrices, the red cell will be present.



How to find the number of TL cells and BR cells in which (i,j) is part of.

	0	1	2	3	4	5	6	7
0	✓	✓	✓	✓	✓			
1	✓	✓	✓	✓	✓			
2	✓	✓	✓	✓	✓			
3	✓	✓	✓	✓	✓	✓	✓	✓
4					✓	✓	✓	✓
5					✓	✓	✓	✓
6					✓	✓	✓	✓
7					✓	✓	✓	✓

TOP LEFT:

rows: [0 i]

cols: [0 j]

total cells = $(i+1) * (j+1)$

BOTTOM RIGHT:

rows: [i N-1]

cols: [j M-1]

total cells = $(N-i) * (M-j)$

Now, to find the total submatrices of which (i,j) is part of -

contribution of (i,j) = TOP LEFT * BOTTOM RIGHT

Every top left cell can be combined with every bottom right cell.

Example

	0	1	2	3	4	5
0	✓	✓	✓			
1	✓	✓	✓			
2	✓	✓	✓	✓	✓	✓
3			✓	✓	✓	✓
4			✓	✓	✓	✓

For (2,2)

TOP LEFT:

$3 * 3 = 9$

BOTTOM RIGHT

$$(5-2) * (6-2) = 3 * 4 = 12$$

Total matrices of which (2,2) is part of $9 * 12$.

Pseudocode

```

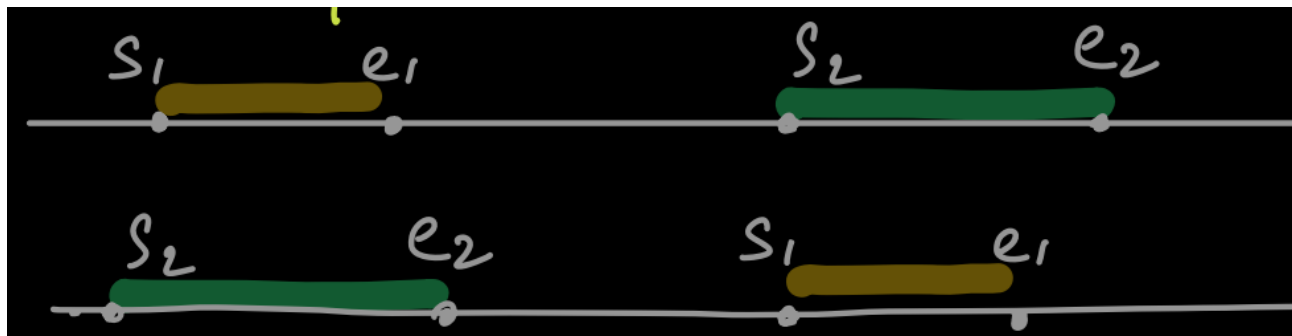
1  total = 0
2  for(int i=0; i<N; i++) {
3      for(int j=0; j<M; j++) {
4
5          top_left = (i+1) * (j+1);
6          bottom_right = (N-i) * (M-j);
7
8          contribution = A[i][j] * top_left * bottom_right;
9
10         total += contribution
11     }
12 }
13 return total

```

Lecture | Advanced DSA: Arrays 3 - Interview Problems**Merge Intervals****Non-Overlapping Condition**

Say there are two Intervals, I1 {s1 e1} & I2 {s2 e2}

Then the condition for them to not overlap is -



```
if(s2 > e1 || s1 > e2)
```

So, if above condition is not followed, it says that Intervals are overlapping!

How to merge overlapping Intervals ?

[I1.start , I1.end] & [I2.start , I2.end]

After merging -

[min(I1.start, I2.start) , max(I1.end, I2.end)]

Problem 1 Merge sorted Overlapping Intervals

Given a sorted list of overlapping intervals, sorted based on start time, merge all overlapping intervals and return sorted list.

Input:

Interval[] = {(0,2), (1,4), (5,6), (6,8), (7,10), (8,9), (12,14)}

Output:

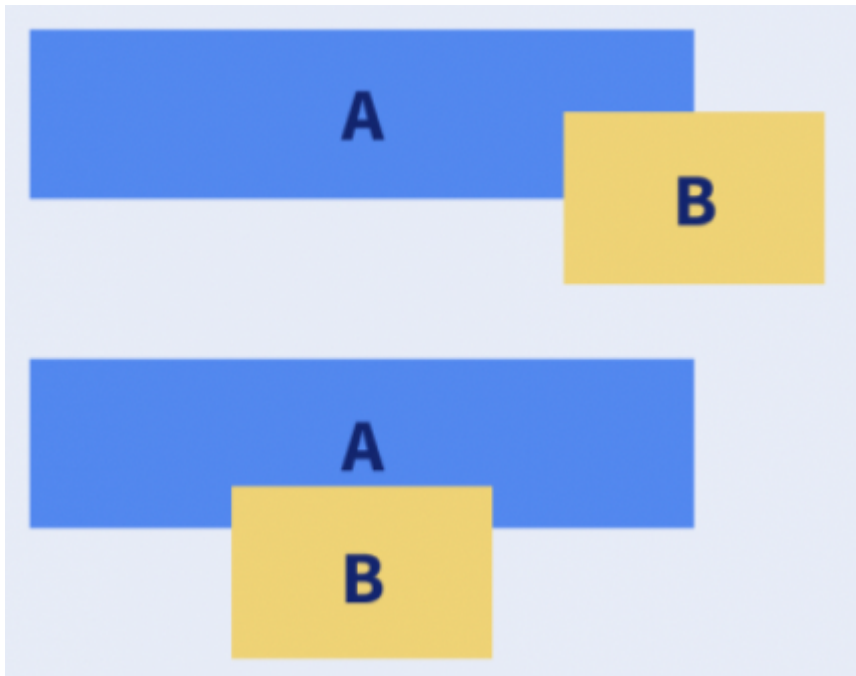
{(0,4), (5,10), (12,14)}

Explanation:

Interval 1	Interval 2		Answer Interval List
0-2	1-4	Overlapping	0-4
0-4	5-6	Not Overlapping	0-4, 5-6
5-6	6-8	Overlapping	0-4, 5-8
5-8	7-10	Overlapping	0-4, 5-10
5-10	8-9	Overlapping	0-4, 5-10
5-10	12-14	Not Overlapping	0-4, 5-10, 12-14

The Array Is Sorted Based on Start Time. What Is the Overlapping Condition?

Say start time of A < start time of B



Overlapping Condition -
If start of B $\leq$ end of A

Approach

- Create an array to store the merged intervals.
- If the current and ith intervals overlaps, merge them. In this case, set merged interval as current interval.
- Else, insert the current interval to answer array since it doesn't overlap with any other interval and update the current Interval to ith Interval.

Pseudocode

```
//Already a class/structure will be present for Interval
//We will only need to create an answer array of type Interval

list<Interval> ans;

// Current Segment
int cur_start = A[0].start, cur_end = A[0].end;

for (int i = 1; i < A.size(); i++) {

    // if i'th interval overlaps with current interval
    if (A[i].start <= cur_end) {
        // merging them
        cur_end = max(cur_end, A[i].end);
    }
    else {
        //adding current interval to answer.
        //create a new Interval
        Interval temp(cur_start, cur_end); //if struct is declared, otherwise
        ans.push_back(temp);

        // update cur Interval to ith
        cur_start = A[i].start;
        cur_end = A[i].end;
    }
}
Interval temp(cur_start, cur_end);
ans.push_back(temp);
return ans;
```

Time and Space Complexity

- TC - $O(N)$
- SC - $O(1)$

Problem 2 - Find First Missing Natural Number

Given an unsorted array of integers, Find first missing Natural Number.

Examples

$\text{arr}[5] = \{3, -2, 1, 2, 7\}$ *ans: 4*

$\text{arr}[7] = \{-9, 2, 6, 4, -8, 1, 3\}$ *ans: 5*

$\{-2, 4, -1, -6, 3, 7, 8, 4, -3\}$ *Ans: 1*

$\{1, 0, -5, -6, 4, 2\}$ *Ans: 3*

$\{1, 2, 5, 6, 4, 3\}$ *ans: 7*

$\{-4, 8, 3, -1, 0\}$ *ans: 1*

$\{4, 2, 1, 3\}$ *ans: 5*

Solution Idea

Can we utilise the fact that answer can be out of 1 to N+1?

If any number other than 1 to N is present, then missing is out of 1 to N only.

If all elements from 1 to N are present, then it will be N+1.

Say we start checking if 1 is present or not, we somehow want to mark the presence of 1.

Now, since we can't use extra space, so we can use indices to mark the presence of a number.

Index 0 can be used to mark presence of 1.

Index 1 can be used to mark presence of 2.

Index 2 can be used to mark presence of 3.

so on...

Now how do we mark the presence ?

One Solution:

We can set element at that index as negative.

But what if negative number is part of the input?

We can just change negative number to a number that is out of our answer range. It can be $N+2$.

$A[] = \{4, 0, 1, -5, -10, 8, 2, 6\}$

Since $N = 8$, answer can only be within range 1 to 9, so we can set $A[i] \leq 0$ as 10

$A[] = \{4, 10, 1, 10, 10, 8, 2, 6\}$

$A[0]=4$, set index 3 as negative

$A[] = \{4, 10, 1, -10, 10, 8, 2, 6\}$

$A[1]=10$ Ignore!

$A[2]=1$, set index 0 as negative

$A[] = \{-4, 10, 1, -10, 10, 8, 2, 6\}$

$A[3]=10$ Ignore!

$A[4]=10$ Ignore!

$A[5]=8$, set index 7 as negative

$A[] = \{-4, 10, 1, -10, 10, 8, 2, -6\}$

$A[6]=2$, set index 1 as negative

$A[] = \{-4, -10, 1, -10, 10, 8, 2, -6\}$

$A[7]=6$, set index 5 as negative

$A[] = \{-4, -10, 1, -10, 10, -8, 2, -6\}$

NOTE: Since we are marking elements as negative, so when checking presence of a certain number, we'll have to consider the absolute value of it.

Pseudocode

```
for(int i = 0; i < N; i++) {  
    if(A[i] <= 0) {  
        A[i] = N + 2;  
    }  
}  
  
for(int i = 0; i < N; i++) {  
    int ele = abs(A[i]);  
  
    if(ele >= 1 && ele <= N) {  
        int idx = ele - 1;  
        A[idx] = -1 * abs(A[i]);  
    }  
}  
  
for(int i = 0; i < N; i++) {  
    if(A[i] > 0) return i + 1;  
}  
return N + 1;
```

Time and Space Complexity

- **TC** - $O(N)$
- **SC** - $O(1)$

Bit Manipulation 1

Truth Table for Bitwise Operators

Below is the truth table for bitwise operators.

a	b	a&b	a b	a^b	~a
0	0	0	0	0	1
0	1	0	1	1	1
1	0	0	1	1	0
1	1	1	1	0	0

Left Shift Operator (<<)

- The left shift operator (<<) shifts the bits of a number to the left by a specified number of positions.
- The left shift operator can be used to multiply a number by 2 raised to the power of the specified number of positions.

In general, it can be formulated as:

$$a \ll n = a * 2^n$$
$$1 \ll n = 2^n$$

Right Shift Operator (>>)

- The right shift operator (>>) shifts the bits of a number to the right by a specified number of positions.
- When we right shift a binary number, the most significant bit (the leftmost bit) is filled with 0.
- Right shift operator can also be used for division by powers of 2.

In general, it can be formulated as:

$$a \gg n = a / 2^n$$
$$1 \gg n = 1 / 2^n$$

Basic AND XOR OR Properties

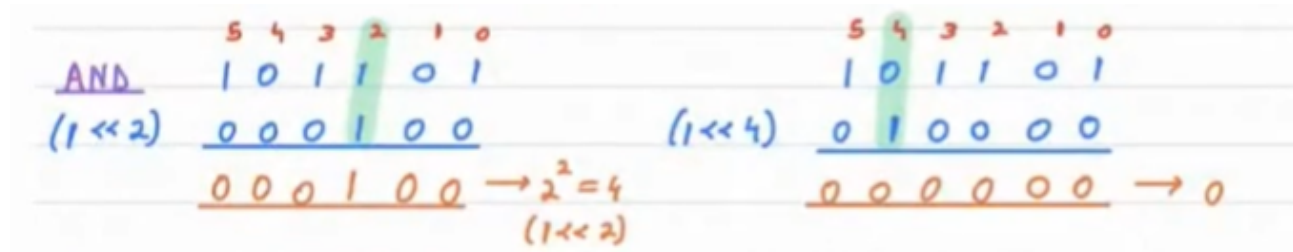
Even/Odd Number

- In binary representation, if a number is even, then its least significant bit (LSB) is 0.
 - **$A \& 1 = 0$** (if A is even)
- Conversely, if a number is odd, then its LSB is 1.
 - **$A \& 1 = 1$** (if A is odd)
- **$A \wedge 0 = A$** (for all values of A)
- **$A \wedge A = 0$** (for all values of A)

Check if ith bit is set or not - AND(&) Operator

$$X = (N \& (1 \ll i))$$

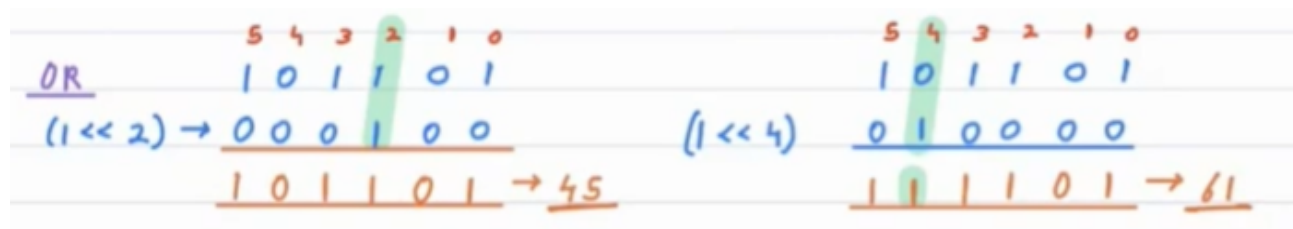
Example



Set ith Bit - OR(|) Operator

$$N = (N | (1 \ll i))$$

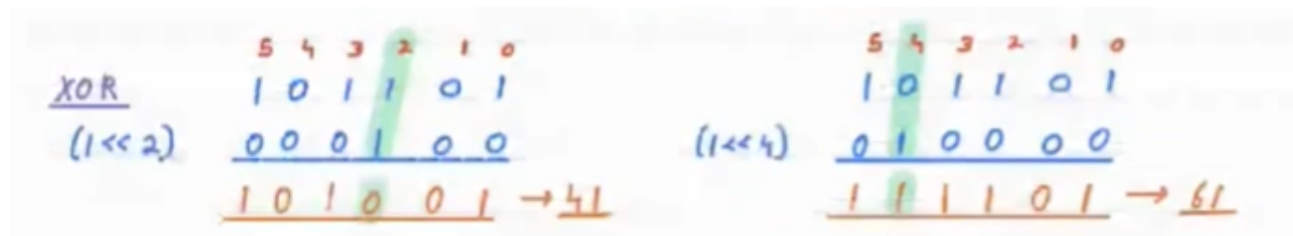
Example



Toggle ith Bit - XOR(^) Operator

$$N = (N \wedge (1 \ll i))$$

Example



UNSET the i^{th} bit of the number **N** if it is set.

```

1  Function unsetbit(int N, int i){
2      int X = (N & (1<<i));
3      if(checkbit(N,i)){
4          N = (N ^ (1<<i));
5      }
6  }
```

Problem - Count the total number of SET bits in **N**

Given an integer **N**, count the total number of SET bits in **N**.

Iterate over all the bits of integer(which is maximum 32) and check whether that bit is set or not. If it is set then increment the answer(initially 0).

```

1  function checkbit(int N, int i){
2      if(N & (1<<i)){
3          return true;
4      }
5      else{
6          return false;
7      }
8  }
9
10 function countbit(int N){
11     int ans = 0;
12     for(i = 0; i < 32; i++){
13         if(checkbit(N, i)){
14             ans = ans + 1;
15         }
16     }
17     return ans;
18 }
```

Here, checkbit function is used to check whether the i^{th} bit is set or not.

Bit Manipulation 2

Problem 1 - Single number 1

We are given an integer array where every number occurs twice except for one number which occurs just once. Find that number.

Example:

Input: [4,5,5,4,1,6,6]

Output: 1

only 1 occurs single time

Approach 1:

Since $\wedge$ helps to cancel out same pairs, we can use it.

Take XOR of all the elements.

Pseudocode

```
1  int x = 0;
2
3  for (int i = 0; i < arr.size(); ++i) {
4      x = x ^ arr[i]; // XOR operation
5  }
6
7  print(x);
```

Approach 2:

Interesting Solution!

Bitwise operators work on bit level, so let's see how XOR was working on bits.

For that, let's write binary representation of every number.

Observations:

For every bit position, if we count the number of 1s the count should be even because numbers appear in pairs, but it can be odd if the bit in a single number is set for that position.

2) Interesting solⁿ.

$A = [2, 3, 5, 6, 3, 6, 2]$

	2 ⁰	2 ¹	2 ²
2 →	0	1	0
3 →	0	1	1
5 →	1	0	1
6 →	0	1	0
3 →	0	1	1
6 →	0	1	0
2 →	0	1	0
	2	6	2
	↓	↓	↓
	1	0	1

OR

0	0	0
0	0	1
0	0	1

Unique no.

Count of set bits on any position

$2 \times (n)$ (Even)
 $2 \times (n) + 1$ (Odd) → Unique no.
 Repeating no's

- We will iterate on all the bits one by one.
- We will count the numbers in the array for which the particular bit is set
- If the count is odd, in the required number that bit is set.

Pseudocode

```
1  int ans = 0;
2
3  for(int i = 0; i < 32; i++) { // go to every bit one by one
4      int cnt = 0;
5
6      for(int j = 0; j < arr.size(); j++) { // iterate on array
7
8          // check if ith bit is set
9          if((arr[j] & (1<<i))cnt++);
10     }
11
12     if(cnt & 1) // If the count is odd
13         ans = ans | 1 << i; // set ith bit in ans
14 }
15
16 print(ans);
```

Time and Space Complexity

- **TC** - $O(N)$
- **SC** - $O(1)$

Similar Question:

Given an integer array, all the elements will occur thrice but one. Find the unique element.

Problem 2 - Single number 3

Given an integer array, all the elements will occur twice except two. Find those two elements.

Input: [4, 5, 4, 1, 6, 6, 5, 2]

Output: 1, 2

Solution Approach:

- Suppose if two unique numbers are **a** and **b**. Their XOR is **c**.
- We will find the position of any set bit in XOR c, it will denote that this bit is different in a and b.
- Now, we divide the entire array in two groups, based upon whether that particular bit is set or not.
- This way a and b will fall into different groups.

- Now since every number repeats twice, they will cancel out when we take XOR of the two groups individually leaving a and b.

Pseudocode

```

1      int xorAll = 0;
2
3      // XOR of all numbers in the array
4      for (int i = 0; i < N; i++) {
5          xorAll ^= A[i];
6      }
7
8      // Find the rightmost set bit position
9      // Note: Any other bit can be used as well
10     int pos;
11
12     for(pos = 0; pos < 32; pos++)
13     {
14         if(checkbit(xorAll,pos))
15             break;
16     }
17
18     num1 = 0;
19     num2 = 0;
20
21     // Divide the array into two groups based on the rightmost set bit
22     for (int i = 0; i < N; i++) {
23         if (checkbit(A[i], pos)) {
24             num1 ^= A[i];
25         } else {
26             num2 ^= A[i];
27         }
28     }
29
30     print(num1);
31     print(num2);

```

Follow Up Question

Given an array A of length N where all the elements are distinct and are in the range [1, N+2]. Two numbers from the range [1, N+2] are missing from the array A. Find the two missing numbers.

Problem 3 - Maximum and pair

Given N array elements, choose two indices(i, j) such that **(i != j)** and **(arr[i] & arr[j])** is maximum.

Input: [5, 4, 6, 8, 5]

Output: [0, 4]

If we take the **&** of 5 with 5, we get 5 which is the maximum possible value here. The required answer would be their respective indices i.e. **0,4**

Observation

1. When bit is set in both the numbers, that bit in their **&** will be 1
2. For answer to be maximum, we will want the set bit to be present towards as left as possible.
3. This indicates that we should start processing the numbers from MSB.

Optimized Solution

- Iterate from the Most significant bit to Least significant bit and for all the numbers in the array, count the numbers for which that bit is set
- If the count comes out to be greater than 1 then pairing is possible, so we include only the elements with that bit set into our vector. Also, set this bit in your answer.
- If the count is 0 or 1, the pairing is not possible, so we continue with the same set and next bit position.

Dry Run

Example: { 26, 13, 23, 28, 27, 7, 25 }

26: 1 1 0 1 0

13: 0 1 1 0 1

23: 1 0 1 1 1

28: 1 1 1 0 0

27: 1 1 0 1 1

07: 0 0 1 1 1

25: 1 1 0 0 1

1. Let's start with MSB, **at position 4**, there are 5 numbers with set bits. Since count is ≥ 2 , we can form a pair. Therefore, in answer 1 will be present at this position.
ans:

1	-	-	-	-
---	---	---	---	---

26:	1	1	0	1	0
13:	0	1	1	0	1
23:	1	0	1	1	1
28:	1	1	1	0	0
27:	1	1	0	1	1
7:	0	0	1	1	1
25:	1	1	0	0	1
	<u>1</u>	-	-	-	-
26:	1	1	0	1	0
13:	0	1	1	0	1
23:	1	0	1	1	1
28:	1	1	1	0	0
27:	1	1	0	1	1
7:	0	0	1	1	1
25:	1	1	0	0	1

2. At position 3, there are 4 numbers with set bits(which haven't been cancelled).
Since count is ≥ 2 , we can form a pair. Therefore, in answer 1 will be present at this position.

ans:

1	1	-	-	-
---	---	---	---	---

We will remove all numbers where 0 is present.

[23 gets removed or is set to 0]

26:	1	1	0	1	0
13:	0	1	1	0	1
23:	1	0	1	1	1
28:	1	1	1	0	0
27:	1	1	0	1	1
7:	0	0	1	1	1
25:	1	1	0	0	1

3. At position 2, there is 1 number with set bit. Since count is less than 2, we can't form a pair. Therefore, in answer 0 will be present at this position.

ans:

1	1	0	-	-
---	---	---	---	---

We will NOT remove any number.

4. At position 1, there are 2 numbers with set bits. Since count is ≥ 2 , we can form a pair. Therefore, in answer 1 will be present at this position.

ans:

1	1	0	1	-
---	---	---	---	---

We will remove all numbers where 0 is present.

[28 and 25 gets removed or are set to 0]

26:	1	1	0	1	0
13:	0	1	1	0	1
23:	1	0	1	1	1
28:	1	1	1	0	0
27:	1	1	0	1	1
7:	0	0	1	1	1
25:	1	1	0	0	1

5. At position 0, there is 1 number with set bit. Since count is < 2 , we can't form a pair. Therefore, in answer 0 will be present at this position.

ans:

1	1	0	1	0
---	---	---	---	---

We will NOT remove any number.

We are done and answer final answer is present in variable ans.

Pseudocode

```
1  int ans = 0;
2
3  for(int i = 31; i >= 0; i--)
4  {
5      //count no. of set bits at ith index
6      int count = 0;
7
8      for(int j = 0; j < n; j++) {
9          if(arr[j] & (1 << i))
10             cnt++;
11     }
12
13     //set that bit in ans if count >=2
14     if(count >= 2) {
15         ans = ans | (1 << i);
16
17         //set all numbers which have 0 bit at this position to 0
18         for(int j = 0; j < n; j++) {
19             if(arr[j] & (1 << i) == 0)
20                 arr[j] = 0;
21         }
22     }
23 }
24
25
26 print(ans);
27
28 //The numbers which cannot be choosen to form a pair have been made zero
29
```

Time and Space Complexity

- **TC** - $O(N)$
- **SC** - $O(1)$

Follow Up Questions

1. Find maximum & of triplets then we will do for count ≥ 3 and for quadruples as count ≥ 4 and so on ...
2. Calculate the Count of Pairs for which bitwise & is maximum
 - Do exactly as above and then traverse on the array and find the number of elements which are greater than 0. Required answer will be N^2 or $N(N-1)/2$